PRACTICAL 6:

Practical6(1)

Problem Phase:

A cafeteria stores clean trays in a fixed-capacity stack. Trays are added using PUSH and removed using POP. The program displays the current top tray after each operation and reports errors when the stack is full or empty.

Solution:

The variable top tracks the top tray. If top == n - 1, the stack is full; if top == -1, it is empty. PUSH adds a tray, POP removes the top tray, and invalid operations display an error without changing the stack.

Practical6(2)

Problem Phase:

A web browser stores visited pages in a stack. The user can visit a new page or press the back button to return to the previous page. The stack grows dynamically without a fixed capacity. The program displays the current page after each operation.

Solution:

Unlike Problem 1, which uses a fixed-size array, this solution uses stack<string>, which grows dynamically as pages are visited. When the user presses Back on an empty history, the program displays an error message and keeps the current history unchanged.

Practical6(3)

Problem Phase:

A calculator receives an infix arithmetic expression, where operators appear between operands. The program converts the expression into postfix notation using a stack and displays the result.

Example: A+B*C → ABC*+

Solution:

The stack temporarily stores operators and controls their precedence. When a closing bracket) is encountered, operators are popped from the stack and added to the postfix expression until the opening bracket (is found. Both brackets are removed, and the remaining operators are processed according to precedence.